

1/29/2025	9 AM – 10 AM	Create Console Program in C#
-----------	--------------	------------------------------

9 AM - 10 AM	Create C# Console Program	Create Console Program in C# Classes in C#
10 AM - 11 AM	Create C# Console Program	Namespaces in C# Variables and Data Types in C#
11 AM - 12 PM	Create C# Console Program	Nullable types in C# Value types & Reference types
12PM - 1 PM	Create C# Console Program	Declare and implement interfaces in C# Operators in C#
1PM - 2 PM		
2 PM - 3 PM	C# Console Program	Partial Classes in C# Static types in C#
3 PM - 4 PM	C# Console Program	Exception Handling, File Operation
4 PM - 5 PM	C# Console Program	"Events Collections, Conditions & Loops Entity Framework Core LINQ"

Console Program: Introduction and Creation

A **console program** is a type of application that interacts with the user through a text-based interface, typically using the command line or terminal. Unlike graphical user interface (GUI) applications, console programs don't have windows, buttons, or graphics. Instead, they rely on text input and output.

Console programs are often used for simple tasks like calculations, data processing, or system management. They are typically written in languages like C, C++, Java, Python, or C# and run in a terminal or command prompt.

Key Features of a Console Program:

1. **Text-based interface:** Interaction is done via keyboard inputs and textual output.

2. **Minimal system resources:** They are lightweight and typically do not consume much memory or processing power.
 3. **Use of standard input/output:** Input is received through the console, and output is printed to the console.
-

Steps to Create a Basic Console Program

Below, I'll walk you through creating a simple console program in **C#** (but similar steps can be applied in other languages like Python, Java, etc.).

1. Create a Console Program in C#:

Step 1: Open Visual Studio (or any code editor)

If you are using Visual Studio, you can create a new project for a Console Application:

- Go to **File > New > Project**.
- Select **Console App (.NET Core)** or **Console App (.NET Framework)**.
- Name your project and click **Create**.

Step 2: Write the Code

Here is an example of a simple console program in C# that takes user input, performs a basic operation, and outputs the result.

```
using System;

namespace ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            // Display a message to the user

            Console.WriteLine("Welcome to the Console Program!");
```

```

// Ask for the user's name

Console.Write("Please enter your name: ");

string name = Console.ReadLine();

// Ask for the user's age

Console.Write("Please enter your age: ");

int age = Convert.ToInt32(Console.ReadLine());

// Calculate the year the user was born

int birthYear = DateTime.Now.Year - age;

// Output the result

Console.WriteLine($"Hello, {name}! You were born in {birthYear}.");


// Wait for user input before closing

Console.WriteLine("Press any key to exit...");

Console.ReadKey();

}

}

}

```

Explanation of Code:

Namespace is a container that holds classes, interfaces, structs, enums, and other namespaces to organize the code logically and avoid name conflicts. It is used to group related types together, providing a way to manage and categorize them in a large application, making the code more readable and maintainable.

```

namespace ConsoleApp {

// Namespace (Nested Namespaces)

// Classes: Program

    Method: Main

// Interfaces

```

```
// Structures  
// Delegates  
}
```

- **Console.WriteLine():** This function prints text to the console.
- **Console.ReadLine():** This function reads user input as a string.
- **Convert.ToInt32():** Converts a string input into an integer (in this case, the user's age).
- **DateTime.Now.Year:** Gets the current year, which is used to calculate the user's birth year.
- **Console.ReadKey():** Waits for a key press before closing the program.

Step 3: Run the Program

1. Press **F5** in Visual Studio or run the program from the terminal to execute it.
2. The program will prompt you to enter your name and age.
3. It will then calculate your birth year and display the result.

1/27/2025	4 PM - 5 PM	Classess in C#
-----------	-------------	----------------

Classes in C#

In C#, a **class** is a blueprint for creating objects (instances). It defines the properties (variables), methods (functions), and events that an object can have. Classes are fundamental to Object-Oriented Programming (OOP), which is the paradigm used by C#.

A class is used to encapsulate related data and functions that can be reused. It provides a structure for objects and defines how they will behave when created.

Key Concepts of a Class in C#

1. **Class Declaration:** A class is declared using the class keyword, followed by the class name.

```
public class Car
{
    // Properties, methods, constructors, etc. go here
}
```

2. **Properties:** Properties are variables that hold the data of the object. They define the state of the object. In C#, properties can have both get and set accessors.

```
public class Car
{
    public string Model { get; set; }
    public string Color { get; set; }
    public int Year { get; set; }
}
```

3. **Methods:** Methods define the behavior of the object. They are functions that can manipulate the object's data or perform actions.

```
public class Car
{
    public string Model { get; set; }
    public string Color { get; set; }
    public int Year { get; set; }

    // Method to display car details
    public void DisplayDetails()
    {
        Console.WriteLine($"Model: {Model}, Color: {Color}, Year: {Year}");
    }
}
```

4. **Constructors:** A constructor is a special method that is called when an object is created from a class. It initializes the object with specific values.

```
public class Car
{
    public string Model { get; set; }
    public string Color { get; set; }
    public int Year { get; set; }

    // Constructor to initialize a new Car object
    public Car(string model, string color, int year)
    {
        Model = model;
        Color = color;
        Year = year;
    }
}
```

5. **Access Modifiers:** C# provides different access modifiers to control the visibility and accessibility of class members (properties, methods, etc.). Some common access modifiers are:

- **private:** The member is only accessible within the same class.
- **public:** The member is accessible from anywhere.
- **protected:** The member is accessible within the class and derived classes.
- **internal:** The member is accessible within the same assembly (project).
- **protected internal:** The member is accessible within the same assembly and in derived classes.

Modifier	Same Assembly	Other Assembly
----------	---------------	----------------

	Same Class	Other Class	Derived Class	Other Class	Derived Class
Private	Yes	No	No	No	No
Public	Yes	Yes	Yes	Yes	Yes
Protected	Yes	No	Yes	No	Yes
Internal	Yes	Yes	Yes	No	No
Protected Internal	Yes	Yes	Yes	No	Yes

Example of a Class in C#

using System;

namespace CarExample

{

 // Declare the Car class

 public class Car

 {

 // Properties of the class

 public string Model { get; set; }

 public string Color { get; set; }

 public int Year { get; set; }

 // Constructor to initialize a Car object

 public Car(string model, string color, int year)

 {

 Model = model;

 Color = color;

 Year = year;

 }

 // Method to display car details

```

        public void DisplayDetails()
        {
            Console.WriteLine($"Model: {Model}, Color: {Color}, Year: {Year}");
        }
    }

// Main Program
class Program
{
    static void Main(string[] args)
    {
        // Create an object (instance) of the Car class
        Car myCar = new Car("Tesla Model S", "Red", 2023);

        // Call the method to display car details
        myCar.DisplayDetails();
    }
}

```

Explanation of the Code:

- **Class Declaration:** `public class Car { ... }` — This defines a class called Car.
- **Properties:** `public string Model { get; set; }` — Properties are defined inside the class to store the data for each car object (Model, Color, Year).
- **Constructor:** `public Car(string model, string color, int year)` — The constructor initializes the properties when a new object is created.
- **Method:** `public void DisplayDetails()` — The DisplayDetails method prints the car's properties to the console.

- **Object Creation:** `Car myCar = new Car("Tesla Model S", "Red", 2023);` — This line creates an instance of the `Car` class with specified values for `Model`, `Color`, and `Year`.

When you run this program, it will output:

yaml

Model: Tesla Model S, Color: Red, Year: 2023

Types of Members in a Class:

1. Fields:

- Fields are variables that hold data for a class.
- They are typically private, with public properties for accessing them.

```
public class Car
{
    private string model; // Field

    public string Model // Property
    {
        get { return model; }
        set { model = value; }
    }
}
```

2. Properties:

- Properties are members that provide a flexible mechanism to read, write, or compute values for private fields.

```
public class Car
{
    private string model;

    public string Model // Property
    {

```

```
    get { return model; }  
    set { model = value; }  
}  
}
```

3. **Methods:**

- Methods define the behavior of a class. They can accept parameters and return values.

```
public class Car  
{  
    public void StartEngine()  
    {  
        Console.WriteLine("The car engine has started.");  
    }  
}
```

4. **Events:**

- Events are members that provide a way to notify other parts of the program when something happens.

```
public class Car  
{  
    public event EventHandler CarStarted;  
  
    public void StartCar()  
    {  
        // Notify all subscribers  
        CarStarted?.Invoke(this, EventArgs.Empty);  
    }  
}
```

-----Different Type of classes-----

1. Static Class

A **static class** is a class that cannot be instantiated. It can only contain static members (fields, properties, methods, etc.). Static classes are typically used for utility or helper functions that don't require an object instance.

Characteristics:

- Cannot be instantiated (no new keyword).
- Can only contain static members.
- Typically used for utility functions, constants, or globally shared resources.

Syntax:

```
public static class MathHelper
{
    public static int Add(int a, int b)
    {
        return a + b;
    }

    public static int Multiply(int a, int b)
    {
        return a * b;
    }
}
```

Usage:

```
int sum = MathHelper.Add(5, 3); // Directly call the static method without creating an instance.
```

2. Sealed Class

A **sealed class** is a class that cannot be inherited. Once a class is sealed, no other class can derive from it. This is useful when you want to prevent further subclassing and ensure that the class is not extended.

Characteristics:

- Cannot be inherited.
- Can still be instantiated (objects can be created from a sealed class).
- Used to prevent further subclassing when the behavior of the class is final or fixed.

Syntax:

```
public sealed class FinalClass
{
    public void DisplayMessage()
    {
        Console.WriteLine("This is a sealed class.");
    }
}
```

Usage:

```
FinalClass obj = new FinalClass();

obj.DisplayMessage();

// Cannot inherit from FinalClass since it's sealed.
```

3. Abstract Class

An **abstract class** is a class that cannot be instantiated directly. It can contain abstract methods (which must be implemented in derived classes) as well as regular methods. Abstract classes are meant to be base classes for other classes.

Characteristics:

- Cannot be instantiated.
- Can contain abstract methods (methods without implementation).
- Can also contain regular methods with implementation.
- Used to define common behavior for derived classes while leaving some methods to be implemented by subclasses.

Syntax:

```
public abstract class Animal
{
    public abstract void MakeSound(); // Abstract method, must be implemented in derived class.

    public void Eat()
    {
        Console.WriteLine("Eating food.");
    }
}
```

Usage:

```
public class Dog : Animal
{
    public override void MakeSound()
    {
        Console.WriteLine("Bark!");
    }
}
```

```
Dog dog = new Dog();
dog.MakeSound(); // Output: Bark!
dog.Eat(); // Output: Eating food.
```

4. Normal Class

A **normal class** (or just a **class**) is the default type of class. It can be instantiated and can contain both instance and static members. You can inherit from normal classes unless they are explicitly marked as sealed.

Characteristics:

- Can be instantiated.

- Can contain instance members and static members.
- Can be inherited (unless sealed).
- Allows full OOP capabilities.

Syntax:

```
public class Vehicle
{
    public string Make { get; set; }
    public string Model { get; set; }

    public void Drive()
    {
        Console.WriteLine("Driving the vehicle.");
    }
}
```

Usage:

```
Vehicle car = new Vehicle();
car.Make = "Toyota";
car.Model = "Corolla";
car.Drive(); // Output: Driving the vehicle.
```

5. Interface

Though not a class, an **interface** is an important type in C#. An interface defines a contract that classes must implement. It can contain method signatures, properties, events, and indexers, but no implementation.

Characteristics:

- Cannot contain implementation.
- Classes that implement an interface must provide implementations for all of its members.
- Supports multiple inheritance.

Syntax:

```
public interface IDriveable
{
    void Drive();
}
```

```
public class Car : IDriveable
{
    public void Drive()
    {
        Console.WriteLine("Car is driving.");
    }
}
```

Usage:

```
IDriveable myCar = new Car();
myCar.Drive(); // Output: Car is driving.
```

Summary of Key Class Types:

Class Type	Description	Example Use Cases
Static Class	Cannot be instantiated, contains only static members.	Utility classes, helper functions.
Sealed Class	Cannot be inherited.	To prevent subclassing.
Abstract Class	Cannot be instantiated, may contain abstract methods.	Base class for other classes to inherit.
Normal Class	Can be instantiated, can contain instance and static members.	Regular class used to represent objects.
Partial Class	Can be split across multiple files.	Organizing code or auto-generated code.
Interface	Defines a contract that must be implemented by classes.	Implementing common functionality across classes.

Each type of class has its use cases depending on your design requirements in C#.
